



Microprocessor & Computer Architecture (µpCA)

UE24CS251B

Session -
5



Microprocessor & Computer Architecture (μpCA)

UE24CS251B

Branch Instructions &
Addressing Modes

Syntax: B{ <cond>} Label

BL{ <cond>} Label

BX{ <cond>} Rm

B	Branch	Program Counter = Label
BL	Branch & Link	Step1: PC will be copied to R14 the Link Register (LR) before branch is taken. Step2: Program Counter = Label
BX	Branch Exchange	Used for changing ARM to Thumb mode or from Thumb mode to ARM mode.
BLX	Branch Exchange with link	

- Branch Instruction- (Unconditional)

B label

...

label: ...

- Conditional Branch Instruction

MOV R0, #0

loop: ...

ADD R0, R0, #1

CMP R0, #10

BNE loop

- **Exercise- 1:** Program to find the sum of N NUMBERS (use conditional branch).
- Store the result in memory location.

Program

```
. DATA
    A: .WORD
    10, 20, 30, 40, 50, 60, 70, 80, 90, 100
    SUM .WORD 0
. TEXT
LDR R1, =A
LDR R2, =SUM
MOV R4, #0      ; INITIALIZATION
MOV R5, #1      ; COUNT register
```

```
L1:    LDR R3, [ R1]
        ADD R4, R4, R3      ; Add next element in the
                                array.
        ADD R1, R1, #4      ; address to the next
                                data
        ADD R5, R5, #1      ; increment the count
                                register
        CMP R5, #11         ; Check whether all
                                numbers are added
        BNE L1              ; Else branch to
```

L1 location

CONDITIONAL BRANCH OPTIONS

Branch	Interpretation	Normal uses
B BAL	Unconditional Always	Always take this branch Always take this branch
BEQ	Equal	Comparison equal or zero result
BNE	Not equal	Comparison not equal or non-zero result
BPL	Plus	Result positive or zero
BMI	Minus	Result minus or negative
BCC	Carry clear	Arithmetic operation did not give carry-out
BLO	Lower	Unsigned comparison gave lower
BCS	Carry set Higher	Arithmetic operation gave carry-out
BHS	or same	Unsigned comparison gave higher or same
BVC	Overflow clear	Signed integer operation; no overflow occurred
BVS	Overflow set	Signed integer operation; overflow occurred
BGT	Greater than	Signed integer comparison gave greater than
BGE	Greater or equal	Signed integer comparison gave greater or equal
BLT	Less than	Signed integer comparison gave less than
BLE	Less or equal	Signed integer comparison gave less than or equal
BHI	Higher	Unsigned comparison gave higher
BLS	Lower or same	Unsigned comparison gave lower or same

Program to find the sum of N numbers using half word

. DATA

```
A:      . HWORD      0x10, 0x20, 0x30, 0x40, 0x50, 0x60, 0x70,
                                0x80, 0x90,  0x0100
```

SUM . WORD 00

. TEXT

LDR R1, =A

LDR R2, =SUM

```
MOV R4, #0 ; I N I T I A L I S A T I O N
```

```
MOV R5, #1      ; COUNT
```

L1: LDRH R3, [R1]

ADD R4, R4, R3

ADD R1, R1, #2

```
ADD R5, R5, #1
```

CMP R5, #11

BNE L1

STRH R4, [R2]

SW 0X011

Addressing Byte Data

Program to find the sum of N numbers using **Byte Data**

; SUM OF N NUMBERS

; DATA GIVEN

. DATA

A: . **BYTE** 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

SUM . word 0

. TEXT

LDR R1, =A

LDR R2, =SUM

MOV R4, #0 ; INITIALISATION

MOV R5, #1 ; COUNT

L1: **LDRB** R3, [R1]

ADD R4, R4, R3

ADD R1, R1, #1

ADD R5, R5, #1

CMP R5, #11

BNE L1

STRB R4, [R2]

SW 0X011

Addressing memory locations

- Memory is addressed by **a register** and **an offset**.

LDR R0, [R1] @ mem[R1]

- Three ways to specify offsets:

- Immediate

LDR R0, [R1, #4] @ mem[R1+4]

- Register

LDR R0, [R1, R2] @ mem[R1+R2]

- Scaled Register

LDR R0, [R1, R2, LSL #2] @ mem[R1+4*R2]

Addressing Modes of ARM Architecture

1. Preindexing or Preindexing without writeback
2. Preindexing with Writeback or Autoindexing
3. Post indexing

Syntax of Addressing Modes used in ARM Architecture

- Pre Indexing Without Write Back

Syntax: **LDR Rd, [Rn,**

OFFSET]

- Pre Indexing With Write Back

Syntax: **LDR Rd,**

[Rn, OFFSET] !

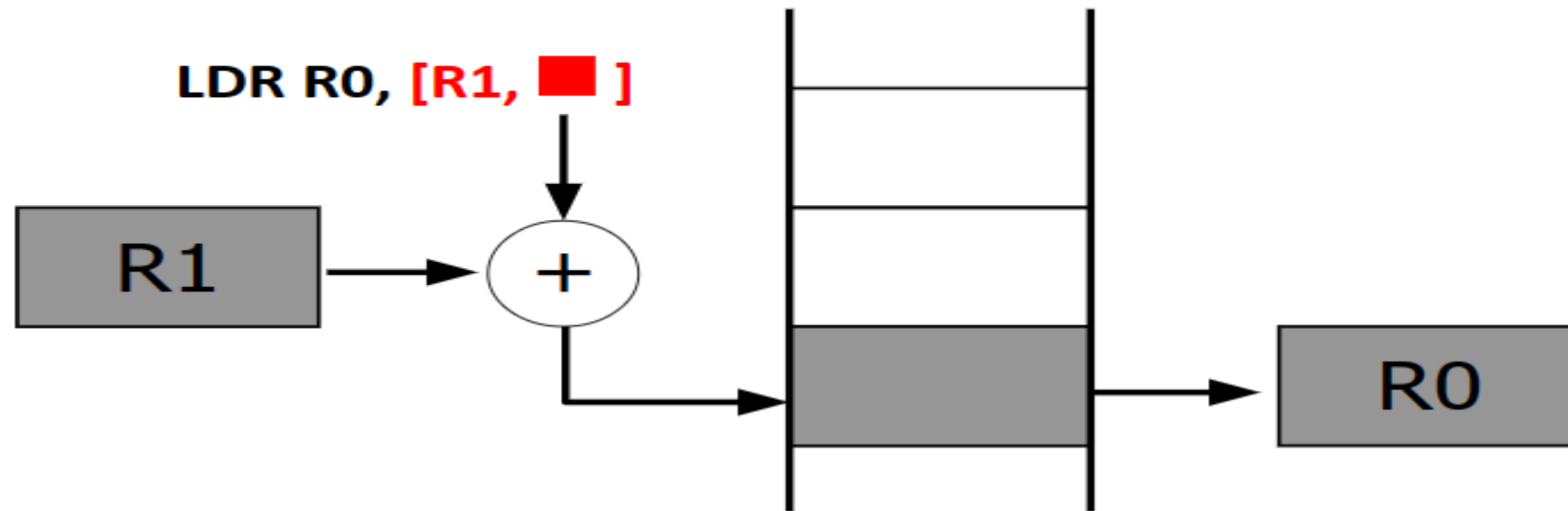
- Post Indexing

- LDR R0, [R1], R2 @ R0=mem[R1]
@ R1=R1+R2**

Will improve the coding style.

Pre-index addressing

LDR R0, [R1, #4] @ R0=mem[R1+4]
@ R1 unchanged



Example – Using preindexing addressing mode

- Program to find the sum of N NUMBERS
- Store the result in memory location.

Program

```
.DATA
    A: .WORD
    10, 20, 30, 40, 50, 60, 70, 80, 90, 100
    SUM .WORD 0
.TEXT
LDR R1, =A
LDR R2, =SUM
MOV R4, #0      ; INITIALIZATION
MOV R5, #1      ; COUNT register
```

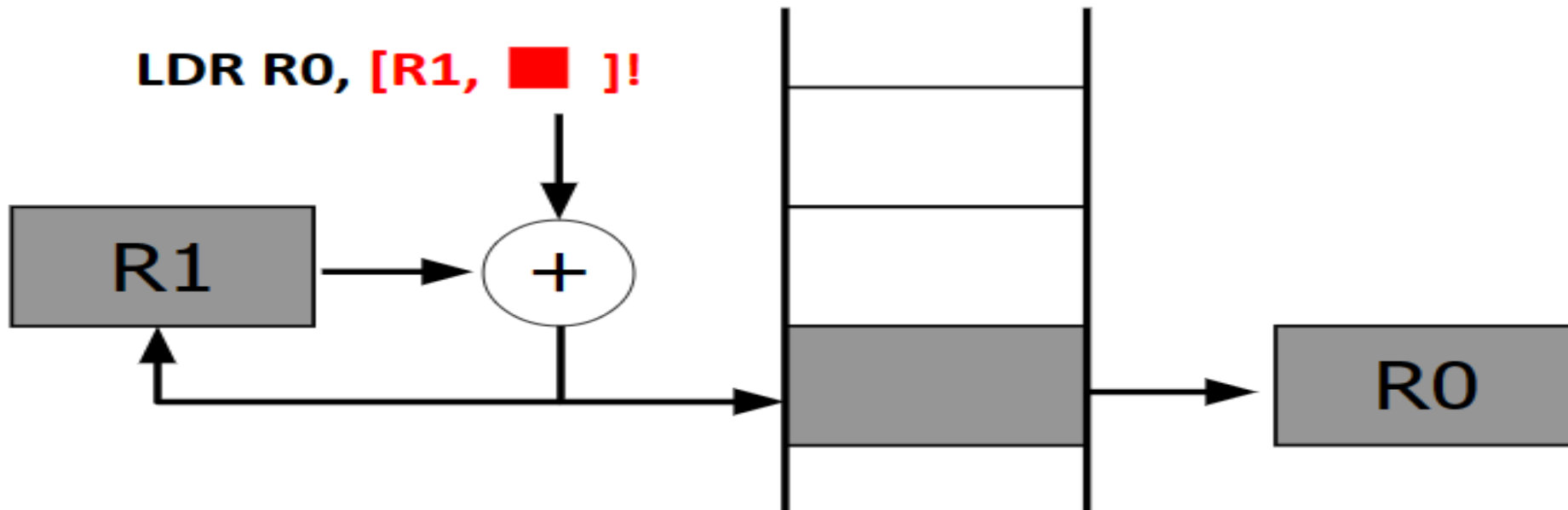
Has no impact on the preindexing mode....

```
                                LDR R3, [ R1]
                                ADD R4, R4, R3
L1:  LDR R3, [ R1, #4]
                                ADD R4, R4, R3      ; Add next element in the
                                array.
                                ADD R1, R1, #4      ; useless
                                ADD R5, R5, #1      ; increment the
                                count register
                                CMP R5, #10         ; Check whether all
                                numbers are added
                                BNE L1              ; Else branch to L1
```

Auto-indexing addressing or Pre-indexing with writeback

**LDR R0, [R1, #4]! @ R0=mem[R1+4]
@ R1=R1+4**

No extra time; Fast;



- Program to find the sum of N NUMBERS
- Store the result in memory location.

Program

. DATA

A: . WORD

10, 20, 30, 40, 50, 60, 70, 80, 90, 100

SUM . WORD 0

. TEXT

LDR R1, =A

LDR R2, =SUM

MOV R4, #0 ; INITIALISATION

MOV R5, #1 ; COUNT register

LDR R3, [R1]

ADD R4, R4, R3

L1: LDR R3, [R1, #4] !

ADD R4, R4, R3 ; Add next element in

the array.

ADD R5, R5, #1 ; increment the count

register

CMP R5, #10 ; Check whether all

numbers are added

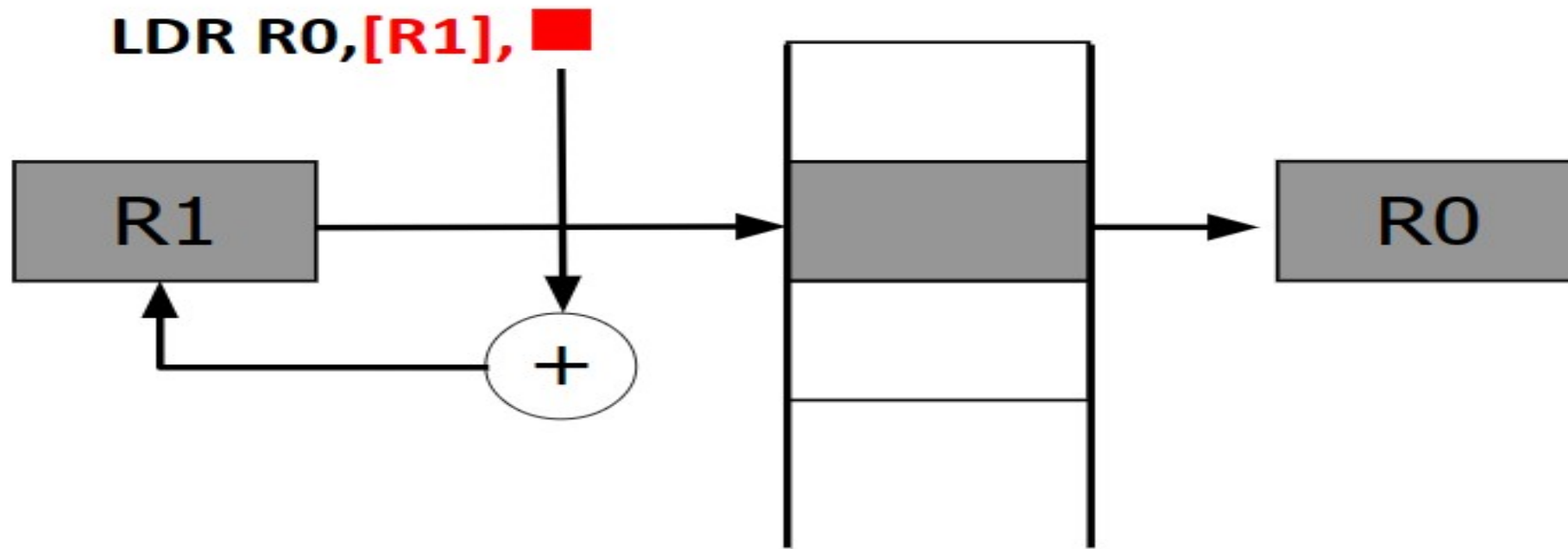
BNE L1 ; Else branch

to L1 location

STR R4, [R2] ; store the

Post-index addressing

LDR R0, [R1], #4 @ R0=mem[R1]
 @ R1=R1+4



- Program to find the sum of N NUMBERS and Store the result in memory location.

Program

```

. DATA
    A: . WORD 10, 20, 30, 40, 50, 60, 70, 80, 90, 100
    SUM . WORD 0
. TEXT
LDR R1, =A
LDR R2, =SUM
MOV R4, #0      ; I N I T I A L I S A T I O N
MOV R5, #1      ; C O U N T   r e g i s t e r

                                L1:  LDR R3, [ R1], #4
                                ADD R4, R4, R3
                                ADD R5, R5, #1
                                CMP R5, #11
                                BNE L1
                                STR R4, [ R2]
                                SW  0X011

```

1. Which ARM instruction is specifically used to branch to a subroutine and save the return address?

- A. B
- B. BX
- C. BL
- D. BLX

2. In the instruction `LDR R0, [R1, R2, LSL #2]`, what does the effective memory address correspond to?

- A. $R1 + R2$
- B. $R1 + (R2 \ll 2)$
- C. $R2 + (R1 \ll 2)$
- D. $R1 \ll (R2 + 2)$

1. Which ARM instruction is specifically used to branch to a subroutine and save the return address?

- A. B
- B. BX
- C. BL
- D. BLX

Answer - BL

2. In the instruction `LDR R0, [R1, R2, LSL #2]`, what does the effective memory address correspond to?

- A. $R1 + R2$
- B. $R1 + (R2 \ll 2)$
- C. $R2 + (R1 \ll 2)$
- D. $R1 \ll (R2 + 2)$

Answer - $R1 + (R2 \ll 2)$

3. Which addressing mode updates the base register after the memory access is completed?

- A. Pre-indexing without writeback
- B. Pre-indexing with writeback (auto-indexing)
- C. Post-indexing
- D. Register-offset addressing

4. Which instruction is most appropriate for loading byte-sized data from memory in ARM?

- A. LDR
- B. LDRH
- C. LDRB
- D. STRB

3. Which addressing mode updates the base register after the memory access is completed?

- A. Pre-indexing without writeback
- B. Pre-indexing with writeback (auto-indexing)
- C. Post-indexing
- D. Register-offset addressing

Answer - Post-indexing

4. Which instruction is most appropriate for loading byte-sized data from memory in ARM?

- A. LDR
- B. LDRH
- C. LDRB
- D. STRB

Answer - LDRB

THANK YOU



Team MPCA

Department of Computer Science and
Engineering